nm

COURSE CODE: 24SDCS02 / 24SDCS02E / 24SDCS02P / 24SDCS02L FULL STACK APPLICATION DEVELOPMENT

#SKILL - 5 ❾ Spring Autowiring Demo using @Autowired

Prerequisites:

- **General Idea on Spring Framework**
- **Modules of Spring Framework**
- **Understanding of Autowiring concepts**

A college application maintains **student** information along with their **certification details**. The **Student** object contains a **Certification** object, and instead of creating this Certification object manually, Spring should automatically provide it using the **@Autowired** annotation. When the required classes are marked with **@Component**, Spring creates their objects and manages them inside the **IoC container**. **Using @Autowired**, the Certification object is supplied to the **Student** object by Spring without using the **new keyword**, allowing both objects to be linked through **annotation-based configuration.**

Tasks:

1. Create a **Certification** class with fields such as id, name, and dateOfCompletion.
2. Create a **Student** class with fields such as id, name, gender, and a Certification object.
3. Annotate both classes with **@Component** to enable Spring detection.
4. Use **@Autowired** on a field, constructor, or setter inside the Student class to inject the Certification object.
5. Configure component scanning using either:
   a. An XML configuration file
   b. A Java configuration class with @Configuration and @ComponentScan
6. Load the Spring IoC container using ApplicationContext.
7. Retrieve the Student bean and print all details, including the injected Certification object.
8. **Push the Spring project to GitHub.**

T E A M FS AD

Appconfig:

```
package com.klu.config;

import org.springframework.context.annotation.ComponentScan; import
org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan(basePackages="com.klu.model")
public class AppConfig {


}
```

MainApp

```
package com.klu.main;

import org.springframework.context.ApplicationContext; import
org.springframework.context.support.ClassPathXmlApplicationContext;

import com.klu.model.Order;
import com.klu.model.Product;

public class MainApp {     public static
void main(String[] args) {

      ApplicationContext ctx =
            new ClassPathXmlApplicationContext("applicationContext.xml");

      Product p1 = (Product) ctx.getBean("p1");
      p1.display();

      Product p2=(Product) ctx.getBean("p2");
      p2.display();

      Order o1 = (Order) ctx.getBean("o1");
      o1.display();
   }
}
```

Products

```
package com.klu.model;

public class Order {

   private int orderId;   private
String customerName;   private
int quantity;
```

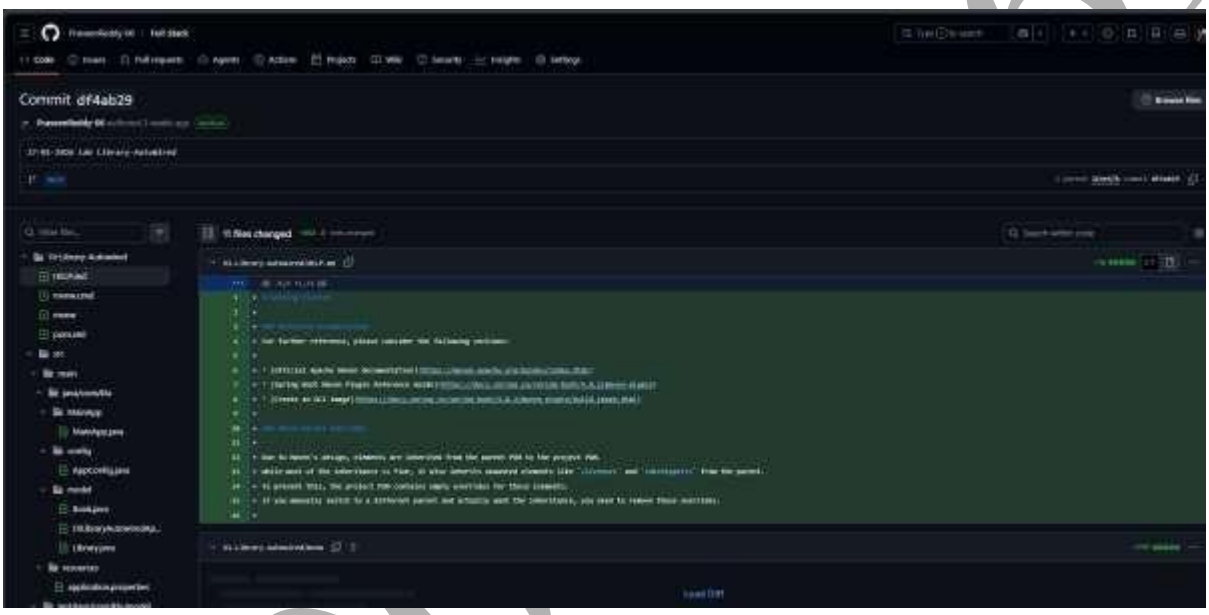
```
    private Product product;

    public Order(int oi, String cn, int q) {
        this.orderId = oi;
this.customerName = cn;
        T E Athis M.quantity = q; FS AD
    }

    public void setProduct(Product product) {
        this.product = product;
    }

    public void display() {
        System.out.println("Order  Details:");
        System.out.println("Order  ID       : " + orderId);
        System.out.println("Customer Name : " + customerName);
        System.out.println("Quantity       : " + quantity);

        System.out.println("Product  Details:");
        System.out.println("Product ID   : " + product.getProductId());
        System.out.println("Product Name : " + product.getProductName());
        System.out.println("Category      : " + product.getCategory());
        System.out.println("Price         : " + product.getPrice());
    }
}
```

Products

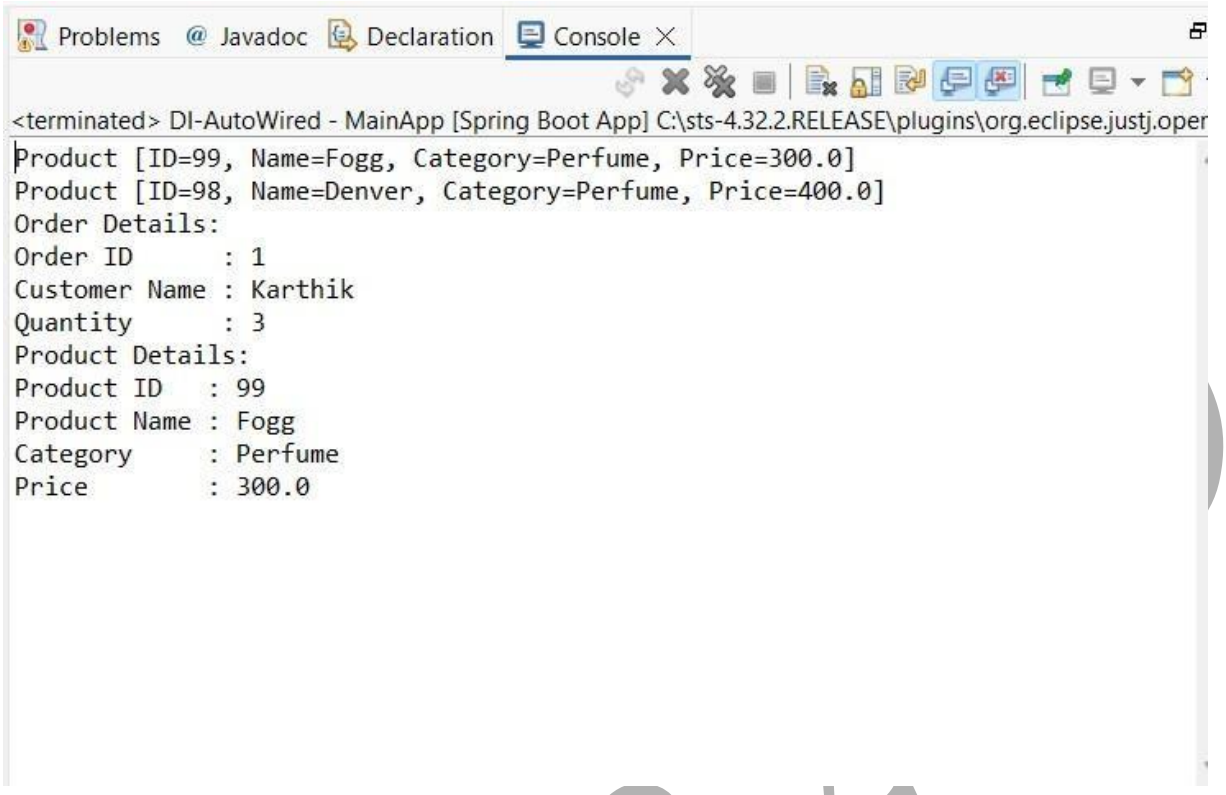
```
package com.klu.model;

public class Product {

    private int productId;
private String productName;
private String category;
    private double price;

    public Product(int pid, String pn, String c, double p) {
        this.productId = pid;
this.productName = pn;
this.category = c;        this.price
= p;
    }

    public int getProductId() {
        return productId;
    }

    public String getProductName() {
        return productName;
    }
```

```
    public String getCategory() {
return category;
    }

    public double getPrice() {
        return price;
    }

    public void display() {
        System.out.println("Product [ID=" + productId + ", Name=" + productName +
T E A M FS AD
                ", Category=" + category + ", Price=" + price + "]");
    }
}
```

GITHUB



OUTPUT



GitHub Link : https://github.com/Anil1843/fsad-skill

VIVA QUESTIONS:

1. What is autowiring in Spring?

Autowiring is a feature in the Spring Framework that automatically injects required dependencies into a bean without explicit configuration. Spring resolves and connects collaborating objects at runtime.

2. What is the purpose of the @Autowired annotation?

@Autowired tells Spring to automatically inject a matching bean into a field, constructor, or setter method. It removes the need for manual wiring in configuration files.

---

3. How does Spring choose which bean to inject?

Spring primarily matches beans **by type**.
If multiple beans of the same type exist, Spring then tries to match **by name**. Additional annotations like @Qualifier can be used to specify the exact bean.

---

4. What is component scanning in Spring?

Component scanning is the process where Spring automatically detects classes annotated with stereotypes such as:

- @Component
- @Service
- @Repository
- @Controller

and registers them as beans in the IoC container.

---

5. What happens if more than one bean matches the dependency type?

Spring throws a **NoUniqueBeanDefinitionException** because it cannot decide which bean to inject. This can be resolved by:

- Using @Qualifier to specify the bean
- Marking one bean as @Primary
- Injecting by bean name

Comment of the Evaluator (If Any)	Evaluator's Observation Marks Secured: _____out of _____ EMP ID & Full Name of the Evaluator: Signature of the Evaluator Date of Evaluation: